

Soapy AI Field Manual — Search Technical Reference



Matt Kennedy

A gated, keyword-search-only showcase for a private AI/LLM field manual. This document is the full system reference — architecture, component deep-dives, data reference, and operations. The product intent and requirements live in [overview.md](#) and [prd.md](#).

Section 1: System Overview

The "Soapy AI Field Manual — Search" (or "saifm") lets interested parties search and investigate the breadth of ai related topics covered in a private field manual. Every query returns **coverage first** (the matching chapters and sections, by number and title) and then a few **capped excerpts** as a taste of the prose. There is no AI model in the loop: it is exact keyword search over a single corpus, served behind a password.

Purpose: demonstrate the manual's *coverage and depth* while keeping the manuscript private. The headline result is the manual's table-of-contents structure; the prose is rationed by design.

In-Short: one corpus, one index, one gate. The full manuscript never reaches the browser or the public repository — searches return only coverage labels and a handful of hard-capped, term-highlighted excerpts, and a gated "Manual Preview" tab offers a single sample chapter.

Design Principles

- **Not an AI app** — no LLM, no embeddings, no generated answers or summarization. SQLite FTS5 keyword search only. The simplest tool that meets the goal is itself the design statement.
- **The corpus is the one secret** — the manuscript and any built index are never committed to the public repo. Everything else (code, docs) is public by design.
- **Coverage first** — results lead with chapter/section structure (the strongest, most copy-resistant summary), then a capped taste of prose.
- **Server-side only, hard caps** — only the gate result and capped excerpts reach the client: ≤ 700 chars/excerpt, ≤ 5 excerpts/query, no browse, no pagination, no "show full section."
- **Gate before content** — no search path returns anything before the password check passes in `session_state`.
- **Build-once, in-memory** — the corpus is tiny (one manual), so the index is rebuilt in memory at startup and never stale relative to the snapshot it loaded.

Tool Stack

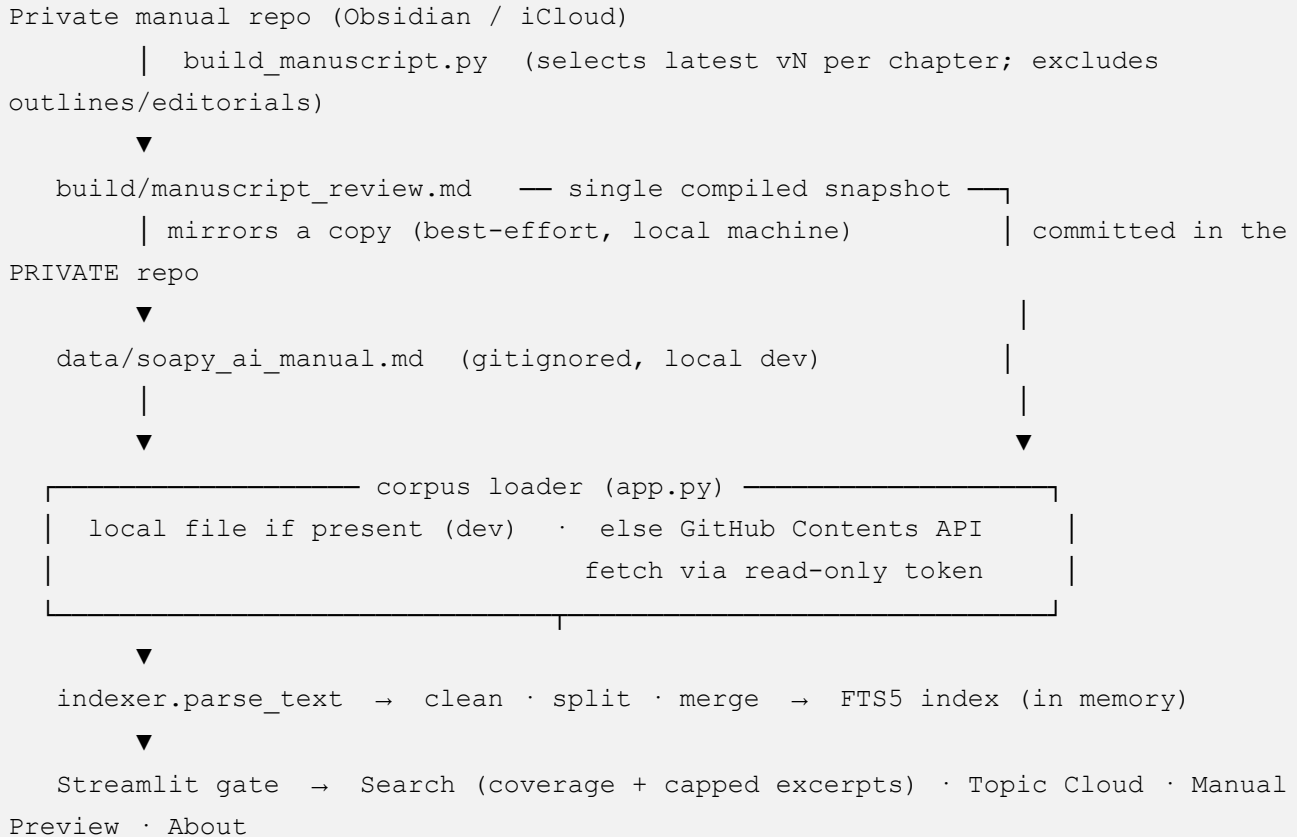
Layer	Tool	Purpose
Language / env	Python ≥ 3.12 via <code>uv</code>	Runtime and dependency management
Search	SQLite FTS5 (stdlib <code>sqlite3</code>)	Full-text index, BM25 ranking, term matching — no external service
UI	Streamlit	Gate + two-layer search UI + Manual Preview
Corpus delivery	local mirror or GitHub Contents API	Local file in dev; private fetch at startup on deploy
Hosting	Streamlit Community Cloud	Free, gated via its secrets manager

No LLM, no API keys for core functionality (a read-only GitHub token is used only to fetch the private corpus on the hosted demo).

Section 2: Architecture & Data Flow

This search is a short, linear flow: a private manuscript is compiled to a single snapshot upstream, delivered to the app (locally or by private fetch), parsed into an in-memory FTS5 index at startup, and served behind a gate.

System Diagram



Setup vs. Runtime

- **Setup (upstream, run on change):** `build_manuscript.py` in the manual repo compiles the current chapters into one snapshot and commits it to the private repo. This is the only step that performs content *selection*; saifm trusts the snapshot.
- **Runtime (per process):** on startup the app loads the snapshot (local or fetched), builds the FTS5 index once (`@st.cache_resource`), and serves queries. There is no per-query setup — each search is an in-memory FTS5 lookup.

Section 3: Component Deep Dives

3.1 Corpus Pipeline — `build_manuscript.py` (manual repo)

Purpose: turn the live, multi-file manuscript into one clean snapshot the app can consume, and deliver it.

Key behavior:

- Selects the **latest vN per chapter** (chapters 01–18 plus Part-0 front matter); excludes `_outline/` `_editorial` files and `sources/`.
- Prepends a `<!-- Source: manual/<dir>/<file> -->` marker before each section and separates sections with a `<div class="chapter-break">` marker.
- Writes `build/manuscript_review.md` (committed in the private repo) and **mirrors a best-effort copy** into `saifm/data/soapy_ai_manual.md` for local dev.

This satisfies the "index only current authored content" requirement at *build time* — saifm never re-implements file discovery.

3.2 Search Engine — scripts/indexer.py

The engine. No Streamlit, no UI — pure parsing and SQLite. `app.py` imports from it.

Parse (`parse_text` / `parse_corpus`). Splits the snapshot into chapters on `<!-- Source ->` markers, derives the chapter number from the path prefix, and splits chapters into sections on `## N.M` headings. Front matter (title page, TOC) and the table-heavy "Tools and Field Cards" sections are excluded.

Clean (`_clean`). Strips search-side noise so excerpts read as prose: HTML wrapper divs, image/attachment markdown, inline emphasis (`** / *`), inline-code backticks, leading list markers, and fenced code blocks. Underscores and real punctuation are preserved.

Merge (`_merge_short`). Folds sub-threshold paragraphs (`< MIN_PARAGRAPH_CHARS = 80`) into a same-section neighbor, so a subheading lead-in becomes part of a contextful paragraph rather than a one-line fragment — this also stops tiny rows from dominating BM25 ranking.

Index (`build_index`). Loads `(chapter, section, paragraph)` rows into an in-memory SQLite FTS5 table (BM25 ranking), shared across Streamlit sessions via `check_same_thread=False`.

Search (`search`). Returns `(coverage, excerpts)` or `None` for a low-signal query:

- `coverage` — distinct matching sections, ranked, deduped by `(chapter, section)`, capped at `MAX_COVERAGE_SECTIONS` (20).
- `excerpts` — for the top matching rows, a centered `~ MAX_EXCERPT_CHARS` (700) window around the first matching term (`_make_excerpt`), all query terms wrapped in highlight markers; one per section, capped at `MAX_EXCERPTS` (5).

- Low-signal queries (empty / stopword-only / single char) are rejected via `query_terms`.

Self-checks. Running `python scripts/indexer.py` validates the parse against [eval_plan.md](#): chapter/section integrity, cleaning residue, the golden-query coverage probe, and low-signal rejection. Exits non-zero on any failure. These checks (plus the output caps and Topic Cloud integrity) are also formalized as a `pytest` suite — see §5.1.

3.3 Gated UI — `scripts/app.py`

The Streamlit front end: a thin gate + presentation layer over `indexer.search`.

- **Gate.** A single shared password from `st.secrets["app_password"]` (or `SAIFM_PASSWORD` env locally), validated in an `on_click` form callback so the gate cleanly disappears on success. Nothing below the gate renders until it passes.
- **Tabs.** `Search` (coverage grouped by chapter, then capped highlighted excerpts), `Topic Cloud` (a keyword-derived coverage map of the manual's concepts — see §3.4), `Manual Preview` (a downloadable sample chapter PDF plus the full Parts/Chapters map), and `About` (the manual's Overview).
- **Highlighting.** Excerpt match markers are HTML-escaped then converted to `<mark>` spans, so terms highlight without markdown collisions.
- **Corpus loading** (`get_corpus_text` / `_preview_bytes`). Reads from local `data/` if present (dev), else fetches from the private manual repo via the GitHub Contents API (`_fetch_private_file`) using a read-only token in secrets. The FTS5 index is built once per process with `@st.cache_resource`.

3.4 Topic Cloud — `scripts/cloud.py`

A second showcase surface on its own gated tab: a coverage map of the manual's concepts. Like search, it is **keyword-derived** (no model) and exposes **no prose** — only aggregate term coverage — so it sits fully inside the content-protection model.

Vocabulary. A curated core of ~26 concept **phrases** (e.g. `context window`, `system prompt`, `golden set`) plus a **sprinkle** of canonical topic **tags** for breadth (`large-language-models`, `fine-tuning`, `ai-deployment`...), with **RAG** promoted as a headline acronym. `SPRINKLE` keys are asserted to be a subset of the shared canonical tag taxonomy (`CANONICAL_TAGS`), so the vocabulary can never drift from it.

Weight (`topic_frequencies`). Each item is weighted by the number of manual sections that cover it — phrases by adjacent-token match over the parsed corpus, tags by alias regex (tags need $\geq$ `MIN_TAG_HITS` sections to appear). Phrases occupy a larger size band than tags so the curated core leads; sqrt scaling keeps the highest-coverage terms from dominating.

Render (`build_html`). A clean descending-size list of styled spans (biggest $\rightarrow$ smallest, soft blue-green, fixed seed), emitted as HTML and shown via `st.markdown`. No image library — the tab adds **no dependencies** (an earlier `wordcloud` rendering was dropped as both messier and heavier). Built once per process in `app.py` (`get_cloud_html`).

Section 4: Data Reference

4.1 Corpus Artifact

Property	Value
Local file (dev)	<code>data/soapy_ai_manual.md</code> — gitignored
Fetches file (deploy)	<code>build/manuscript_review.md</code> in the private manual repo (identical content)
Size	~780 KB, ~11,140 lines; Part 0 + chapters 1–18 + glossary + planning examples
Indexed rows	~2,077 paragraphs after cleaning, merging, and section exclusion

4.2 FTS5 Table — sections

Column	Indexed	Role
<code>chapter_num</code>	no	Chapter number (from the <code>Source</code> path)
<code>chapter_title</code>	no	Chapter title (from <code># Chapter N - ...</code>)
<code>section_number</code>	no	e.g. <code>12.7</code> (empty for chapter-intro prose)
<code>section_title</code>	yes	e.g. <i>MCP: Standardized Tool and Resource Integration</i>
<code>body</code>	yes	The cleaned paragraph text

Tokenizer `porter unicode61`. Because every paragraph row also carries its `section_title` as an indexed column, a query matches a section *via its title* even when no single paragraph

contains all terms — which makes the coverage layer robust.

4.3 Format Contract (the decoupling boundary)

`indexer.py` depends on these properties of the snapshot; as long as the manual's builder keeps emitting them, the two projects stay independent:

Element	Used for
<code><!-- Source: manual/<dir>/<file> --></code>	Chapter split + chapter number
<code># / ## / ###</code> headings	Section numbers + titles (the coverage layer)
<code><div class="chapter-break"></code>	Secondary section boundary
Blank-line paragraphs	Excerpt granularity

4.4 Output Caps (content-protection model)

Constant	Value	Effect
<code>MAX_EXCERPTS</code>	5	Max excerpts per query
<code>MAX_EXCERPT_CHARS</code>	700	Centered window per excerpt (context before + after)
<code>MAX_COVERAGE_SECTIONS</code>	20	Max sections listed in coverage
<code>MIN_PARAGRAPH_CHARS</code>	80	Merge threshold for short paragraphs

No browse, no pagination, no "show full section." The "Manual Preview" tab is a deliberate, bounded exception: a single downloadable sample chapter, gated, never committed to the public repo.

4.5 Topic Cloud Vocabulary

Source	Count	Role
Curated phrases (+ <code>RAG</code>)	~27	The headline concepts; larger size band
Canonical tags (sprinkle)	~23 of 62	Topic-level breadth; smaller size band, $\geq$ <code>MIN_TAG_HITS</code> (5) sections to appear

Derived from the same parsed corpus as `search ( indexer.parse_text )` — no new storage or index. Output is aggregate section counts only (no prose), so the tab adds nothing to expose under the content-protection model (§4.4).

Section 5: Operations

This Soapy AI Manual Search runs locally for development and on Streamlit Community Cloud for the gated demo. The corpus and preview PDF are never in the public repo.

5.1 Local Development

```
uv sync                                # set up the venv (Python 3.12)
uv run python scripts/indexer.py       # build + inline self-checks
uv run pytest                          # run the test suite (tests/)
uv run streamlit run scripts/app.py --server.port 8503
```

Local dev needs only `app_password` in `.streamlit/secrets.toml` (gitignored); the corpus and preview load from local `data/`.

Tests. The `tests/` suite (`uv run pytest`) is the formal version of the eval-plan checks: golden coverage, the output caps, query hygiene, parse integrity, and the Topic Cloud's prose-free guarantee — deterministic, no network or model. Session-scoped fixtures build the corpus + index once and skip cleanly if the snapshot is absent. `pytest` is a uv dev-dependency (`[dependency-groups] dev`), kept out of the deploy mirror. See [eval_plan.md](#) §6.

5.2 Deploy — Streamlit Community Cloud

- **Main file:** `scripts/app.py` · **Python:** 3.12.
- **Secrets:** `app_password`, `github_token` (fine-grained PAT, read-only *Contents* on the private manual repo), `manual_repo` (`owner/name`). Optional: `corpus_path`, `preview_path`, `manual_branch`.
- **Corpus delivery:** on startup the app fetches `build/manuscript_review.md` (and the preview PDF) from the private repo via the token — neither file lives in the public repo.
- **Port:** the committed `config.toml` does **not** pin a port; Streamlit Cloud health-checks its default (8501), so pinning breaks the deploy.

5.3 Refresh Cadence

The manual is edited continuously. To update the live demo: rebuild

`build/manuscript_review.md` (and `manual_preview.pdf` if changed), commit them to the **private** repo, then **Reboot** the Streamlit app — it re-fetches and rebuilds the index on startup. The public repo is untouched.

5.4 Common Issues

Symptom	Cause / Fix
Search shows <i>"corpus couldn't be loaded"</i>	Fetch failed — check <code>github_token</code> , <code>manual_repo</code> , and that <code>build/manuscript_review.md</code> is committed in the private repo
Deploy fails health check on <code>:8501</code>	A pinned <code>server.port</code> in <code>config.toml</code> — remove it; pass <code>--server.port</code> only for local runs
Manual Preview shows <i>"preview isn't available"</i>	<code>manual_preview.pdf</code> not present in the private repo — search/coverage still work
Local self-checks fail after a corpus change	The snapshot likely broke the format contract (§4.3) — confirm the builder still emits <code>Source</code> markers and <code>## N.M</code> headings

Interface

The gated UI — landing gate, coverage-first search, capped excerpts, and the Manual Preview tab — is shown in the screenshots in the [README](#).